

Análisis de Fútbol Robótico con Visión por Computadora

Documentación técnica completa — Equipo AJOLOTES FC

AJOLOTES FC — Copa FutBotMX 2026 (Secihti · Meta · CENTRO)

2026-05-21

Índice

Introducción: ¿qué construimos y por qué?	2
1. Conceptos básicos de visión por computadora	3
1.1 ¿Qué es una imagen para una computadora?	3
1.2 El espacio de color HSV	3
1.3 Tres tareas distintas: clasificar, detectar, segmentar	3
1.4 ¿Qué es un "modelo" de inteligencia artificial?	4
1.5 ¿Qué es el seguimiento (tracking)?	4
1.6 ¿Qué es una homografía?	4
2. El contexto del problema: la cancha y los robots	5
3. El modelo base: SAM 3	6
3.1 Cómo funciona SAM 3 por dentro (a grandes rasgos)	6
3.2 Un hallazgo práctico importante	6
3.3 Optimización: media precisión (fp16)	6
4. Arquitectura general del sistema	7
5. El pipeline paso a paso	8
5.1 Ingesta de video	8
5.2 Calibración: encontrar el campo y la homografía	8
5.3 Segmentación con SAM 3	8
5.4 Filtrado de detecciones falsas	8
5.5 Seguimiento (tracking)	9
5.6 Re-identificación de equipos	9
5.7 Proyección a coordenadas del mundo	9
5.8 Detección de eventos	9
5.9 Estadísticas en tiempo real	10
5.10 Visualizaciones	10
6. ¿Qué es fine-tuning y LoRA? (concepto base)	11
6.1 Fine-tuning	11
6.2 LoRA: ajuste fino eficiente	11

6.3 El problema de las etiquetas	11
7. Las cuatro líneas de innovación	12
8. Resultados y validación	13
8.1 Calidad del software	13
8.2 Resultados sobre videos reales	13
9. Cómo reproducir el proyecto	14
10. Glosario de términos	15

Introducción: ¿qué construimos y por qué?

Este documento explica, **desde cero y con conceptos básicos**, el sistema de visión por computadora que el equipo **AJOLOTES FC** desarrolló para la **Copa FutBotMX 2026**, capítulo *Visión por Computadora*, organizada por la Secretaría de Ciencia, Humanidades, Tecnología e Innovación (Secihti) junto con Meta y CENTRO.

El reto es el siguiente: nos dan **videos de partidos de fútbol robótico** (robots pequeños que juegan fútbol sobre una mesa-cancha) y debemos construir un programa que, **de forma automática**, “entienda” lo que pasa en el video:

- dónde está el campo, los robots de cada equipo y el balón;
- a dónde se mueve cada robot y el balón a lo largo del tiempo;
- qué eventos ocurren (un gol, un pase, un choque entre robots);
- estadísticas del partido (quién tuvo más posesión del balón, cuántos goles).

El requisito obligatorio de la competencia es usar como base un modelo de inteligencia artificial llamado **SAM 3** (Segment Anything Model 3), creado por Meta. La categoría **Profesional** —en la que competimos— exige además una **innovación** sobre ese modelo (lo explicamos en la sección 7).

Antes de entrar al sistema, necesitamos entender algunos conceptos básicos.

1. Conceptos básicos de visión por computadora

La **visión por computadora** (en inglés *computer vision*, CV) es la rama de la inteligencia artificial que busca que una computadora “entienda” imágenes y videos de forma parecida a como lo hace un ojo humano con un cerebro detrás.

1.1 ¿Qué es una imagen para una computadora?

Para una persona, una foto es una escena. Para una computadora, una imagen es simplemente una **cuadrícula de números**. Cada celda de esa cuadrícula es un **píxel** (del inglés *picture element*).

- Una imagen de 1920×1080 píxeles tiene 1920 columnas y 1080 filas, es decir, más de 2 millones de píxeles.
- Cada píxel a color guarda **tres números**: cuánto rojo, cuánto verde y cuánto azul tiene (modelo **RGB**). Cada número va de 0 a 255.
- Por ejemplo, el rojo puro es (255, 0, 0); el blanco es (255, 255, 255); el negro es (0, 0, 0).

Un **video** es solo una secuencia rápida de imágenes (llamadas *frames* o cuadros). Los videos del reto van a 30 o 60 *frames* por segundo (fps), es decir, 30 o 60 imágenes por cada segundo.

1.2 El espacio de color HSV

El modelo RGB es bueno para mostrar colores en pantalla, pero es **malo para identificar colores** programáticamente, porque la iluminación cambia los tres canales a la vez. Por eso usamos otro modelo llamado **HSV**:

- **H** (*Hue*, matiz): el color “puro” en un círculo de 0 a 179 (rojo, naranja, amarillo, verde, azul, morado...). Es lo que llamamos coloquialmente “el color”.
- **S** (*Saturation*, saturación): qué tan intenso o “lavado” es el color. Un rojo bandera tiene saturación alta; un rosa pálido, baja.
- **V** (*Value*, brillo): qué tan claro u oscuro es.

La ventaja: si queremos detectar “lo verde de la cancha”, basta con buscar píxeles cuyo **matiz (H)** esté en el rango del verde, sin importar tanto si hay sombra o luz fuerte. Usamos HSV intensamente en este proyecto para detectar el campo, las líneas blancas y las porterías de color.

1.3 Tres tareas distintas: clasificar, detectar, segmentar

Es importante distinguir tres tareas que suenan parecidas:

- **Clasificación**: responder “¿qué hay en esta imagen?” con una etiqueta global. Ejemplo: “esta foto contiene un robot”.
- **Detección de objetos**: poner un **recuadro** (en inglés *bounding box*) alrededor de cada objeto. Ejemplo: “hay un robot aquí, otro allá y el balón acá”, cada uno con su rectángulo.
- **Segmentación**: ir más fino y decir, **píxel por píxel**, a qué objeto pertenece cada uno. En vez de un rectángulo, obtenemos la **silueta exacta** (llamada **máscara**) del objeto.

SAM 3 hace **segmentación**: nos da la máscara precisa de cada robot, del balón y del campo. De esa máscara podemos sacar fácilmente el recuadro y el centro del objeto.

1.4 ¿Qué es un “modelo” de inteligencia artificial?

Un **modelo** de aprendizaje profundo (*deep learning*) es un programa que **aprendió** a hacer una tarea a partir de millones de ejemplos, en lugar de seguir reglas escritas a mano. Internamente es una enorme función matemática con millones de números ajustables llamados **parámetros** o **pesos**.

- **Entrenar** un modelo es ajustar esos pesos mostrándole ejemplos y corrigiéndolo cuando se equivoca.
- **Inferencia** es usar el modelo ya entrenado para hacer predicciones sobre datos nuevos.

SAM 3 ya viene **preentrenado** por Meta sobre una cantidad gigantesca de imágenes, así que sabe segmentar casi cualquier objeto sin que nosotros lo entrenemos. Nosotros solo lo usamos (inferencia) y, como innovación, lo **afinamos** un poco para nuestro dominio (sección 7.2).

1.5 ¿Qué es el seguimiento (tracking)?

La segmentación funciona **frame por frame**: en cada imagen encuentra los objetos, pero no sabe que “el robot del frame 1 es el mismo robot del frame 2”. El **seguimiento** (*tracking*) es la tarea de **asignar una identidad estable** a cada objeto a lo largo del tiempo: el robot #3 sigue siendo el #3 aunque se mueva. Esto nos permite dibujar **trayectorias** y calcular velocidades.

1.6 ¿Qué es una homografía?

La cámara que grabó los videos no está justo arriba de la cancha mirando hacia abajo: está a un lado, en diagonal (es una cámara de espectador). Eso deforma la cancha: lo que en la realidad es un rectángulo, en la imagen se ve como un **trapecio** (los lados lejanos se ven más pequeños).

Una **homografía** es una transformación matemática (una matriz de 3×3) que “endereza” esa perspectiva: convierte las coordenadas de un punto en la imagen deformada a las coordenadas reales sobre la cancha, en milímetros, como si la viéramos desde arriba (vista *top-down* o cenital). Gracias a la homografía podemos decir “el balón está en el milímetro (1200, 800) de la cancha”, y de ahí calcular distancias y velocidades reales.

2. El contexto del problema: la cancha y los robots

Conocer el “campo de juego” físico es clave, porque muchas decisiones del sistema dependen de sus dimensiones exactas (reglamento oficial § 7):

- La cancha mide **219 cm de largo × 158 cm de ancho** (2190 mm × 1580 mm) en su zona de juego interior.
- Está rodeada por **paredes negras** de al menos 22 cm de alto.
- Tiene **líneas blancas** que marcan el perímetro de juego y las áreas, con **escuadras en forma de L** en las cuatro esquinas.
- Las **porterías** son cajas de color: una **amarilla** y una **azul**, cada una de 60 cm de ancho, centradas en los lados cortos.
- El **balón** es una pelota pequeña de color naranja.
- Los **robots** son máquinas autónomas de varios colores (rojo, verde oliva, blanco/plata, según el equipo).

Una complicación importante: los videos fueron grabados por **espectadores** con cámara en mano (iPhone / lentes con IA), no con una cámara fija cenital. Esto significa que:

- la cámara **se mueve** (paneos suaves, ocasional desenfoque por movimiento);
- a veces aparecen **manos o personas** tapando parte del campo;
- la vista es **oblicua** y a veces una esquina del campo queda fuera del cuadro.

Diseñamos todo el sistema para ser robusto a estas condiciones.

3. El modelo base: SAM 3

SAM 3 (*Segment Anything Model 3*) es un modelo de Meta que segmenta objetos en imágenes y video. Su característica más poderosa es que acepta **prompts** (instrucciones) de varios tipos para decirle **qué** segmentar:

- **Prompt de texto:** le escribimos "soccer robot" (robot de fútbol) y nos devuelve las máscaras de todos los robots que encuentra.
- **Prompt de caja (box):** le damos un rectángulo y segmenta el objeto que está dentro.
- **Prompt de punto:** le damos una coordenada y segmenta el objeto en ese punto.

Esto se llama **vocabulario abierto**: no está limitado a una lista fija de categorías, sino que entiende lenguaje natural.

3.1 Cómo funciona SAM 3 por dentro (a grandes rasgos)

SAM 3 está formado por varias piezas (lo verificamos inspeccionando el modelo):

- **Vision encoder** (codificador visual): "lee" la imagen y la convierte en una representación numérica rica. Es la parte más grande (32 capas).
- **Text encoder** (codificador de texto): convierte el prompt de texto ("soccer robot") en números que el modelo entiende.
- **Geometry encoder:** procesa los prompts de tipo caja o punto.
- **Mask decoder** (decodificador de máscaras): combina todo lo anterior y produce las máscaras finales.

Internamente, SAM 3 propone hasta **200 candidatos** (estilo *DETR*) por imagen y para cada uno entrega una máscara, un recuadro y una puntuación de confianza (*score*). Nos quedamos con los candidatos de mayor confianza.

3.2 Un hallazgo práctico importante

Probamos distintos prompts y descubrimos algo contraintuitivo: **los prompts simples funcionan mejor que los elaborados**. El prompt "soccer robot" obtuvo una confianza de 0.94, mientras que un prompt detallado como "small mobile soccer robot with a colored flag" (robot de fútbol pequeño y móvil con una bandera de color) solo obtuvo 0.34. SAM 3 entiende bien el concepto "robot", pero los modificadores largos lo confunden. Por eso usamos prompts cortos y directos.

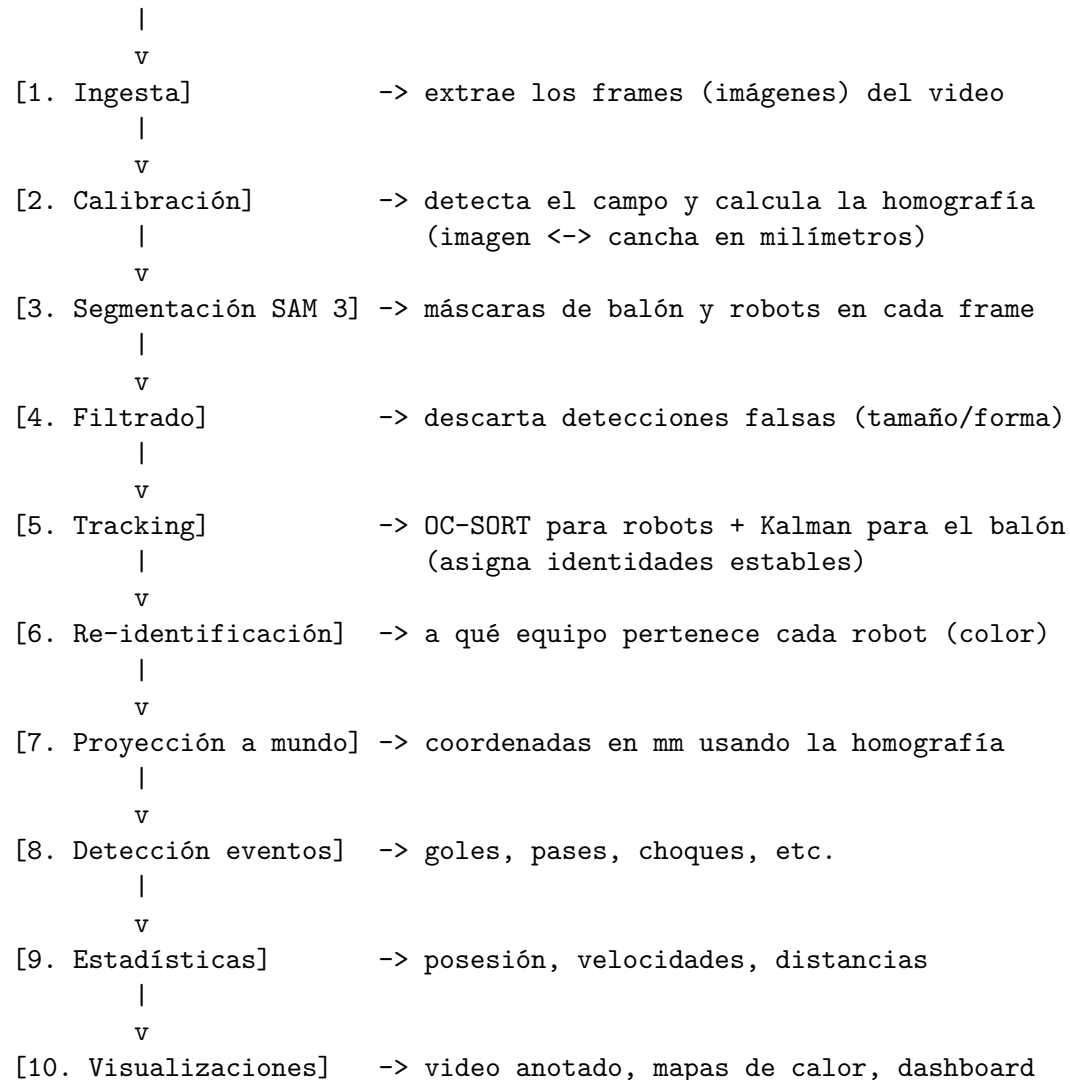
3.3 Optimización: media precisión (fp16)

Los modelos guardan sus números en formatos de distinta precisión. Por defecto usan *float32* (32 bits por número). Nosotros usamos **fp16** (*float16*, 16 bits), que ocupa la mitad de memoria y corre más rápido en la tarjeta gráfica, con prácticamente la misma calidad de detección. Esto bajó el uso de memoria de video (VRAM) y dio cerca de 2.5× de aceleración.

4. Arquitectura general del sistema

El sistema es un **pipeline**: una cadena de etapas donde la salida de una alimenta a la siguiente. Esta es la cadena completa:

Video crudo (30/60 fps)



A continuación explicamos cada etapa con detalle.

5. El pipeline paso a paso

5.1 Ingesta de video

Leemos el video con la librería OpenCV. Para ahorrar cómputo no procesamos todos los frames: usamos un parámetro llamado **stride** (paso). Un *stride* de 5 significa "procesa 1 de cada 5 frames". En un video de 30 fps, eso son 6 frames procesados por segundo, suficiente para seguir el juego y mucho más rápido que procesar los 30.

5.2 Calibración: encontrar el campo y la homografía

Esta etapa fue de las más difíciles. Necesitamos las **4 esquinas reales del campo de juego** (las que marcan las escuadras blancas) para calcular la homografía. Desarrollamos dos métodos complementarios:

1. **Detección por líneas blancas (método principal)**: aislamos los píxeles blancos que están dentro de la zona verde del campo, detectamos las líneas rectas con el algoritmo de **Hough**, las agrupamos por orientación (horizontales vs verticales) y calculamos dónde se cruzan: esas intersecciones son las esquinas.
2. **Envolvente convexa del fieltro verde (método de respaldo)**: si las líneas blancas no se ven bien (por desenfoque o sombras), detectamos toda la mancha verde de la cancha, le calculamos su **envolvente convexa** (el contorno "estirado" que la rodea sin concavidades, lo que elimina los huecos que dejan las manos o personas encima del campo) y la aproximamos a un cuadrilátero de 4 esquinas.

Una vez con las 4 esquinas en la imagen, y sabiendo que en la realidad la cancha mide 2190×1580 mm, OpenCV calcula la **matriz de homografía** que traduce cualquier punto de la imagen a milímetros sobre la cancha.

Además, en esta etapa detectamos las **porterías por color**: buscamos la mancha amarilla y la mancha azul pegadas al borde del campo. Guardamos sus recuadros porque los usamos después para detectar goles de forma confiable.

Como la cámara se mueve, **recalibramos las porterías cada 60 frames** para que el recuadro no se "desfase" del lugar real (esto corrigió un problema de goles detectados de más).

5.3 Segmentación con SAM 3

Para cada frame procesado, le pedimos a SAM 3 dos cosas con prompts de texto: las máscaras del **balón** ("ball") y las de los **robots** ("soccer robot"). SAM 3 nos devuelve las siluetas con su nivel de confianza.

5.4 Filtrado de detecciones falsas

SAM 3 a veces confunde objetos: marca una caja amarilla, una mano o un parche de luz como si fuera un robot. Para limpiar esto aplicamos **filtros geométricos** sobre cada detección de robot:

- **Confianza mínima**: descartamos detecciones con *score* bajo.
- **Área plausible**: el recuadro debe ocupar entre 0.1 % y 5 % del frame (ni un punto diminuto ni media pantalla).

- **Proporción (aspecto):** la relación alto/ancho debe ser razonable (entre 0.4 y 4.0), porque un robot no es ni una línea muy fina ni una mancha cuadrada gigante.

Para el balón, si SAM 3 no lo encuentra, tenemos un **detector de respaldo por color HSV** que busca la pelota naranja directamente.

5.5 Seguimiento (tracking)

Usamos dos algoritmos distintos según el objeto:

- **Robots** → **OC-SORT** (de la librería BoxMOT): un algoritmo de seguimiento multi-objeto que asocia las detecciones de un frame al siguiente usando la posición y el movimiento, asignando a cada robot un **número de identidad (track id)** estable.
- **Balón** → **filtro de Kalman 2D**: el balón es pequeño, rápido y a veces se oculta tras un robot. El **filtro de Kalman** es una técnica matemática que **predice** dónde estará el balón en el siguiente frame basándose en su movimiento previo, y corrige esa predicción cuando vuelve a verlo. Así mantenemos una trayectoria suave incluso con oclusiones breves.

5.6 Re-identificación de equipos

Saber que el robot #3 existe no basta: hay que saber **de qué equipo es**. Como los equipos usan colores distintos, miramos el **color dominante** de cada robot (su matiz y saturación en HSV) y los agrupamos en dos conjuntos con un algoritmo de *clustering* llamado **k-means** (con $k=2$, dos equipos).

Esta parte tuvo un problema sutil que resolvimos: si en los primeros segundos la cámara solo enfoca a un equipo, el sistema “creía” que solo había un color. Lo arreglamos con tres mejoras:

1. **Periodo de calentamiento más largo** (30 frames antes de decidir).
2. **Recálculo continuo** de los colores de equipo a medida que avanza el partido (no una sola vez al inicio).
3. **Votación temporal**: la decisión final del equipo de cada robot se toma por mayoría de sus últimas observaciones, no por una sola, lo que suaviza el ruido.

5.7 Proyección a coordenadas del mundo

Con la homografía de la etapa 2, convertimos el centro de cada robot y del balón de píxeles a **milímetros sobre la cancha**. A partir de aquí, todo el análisis ocurre en coordenadas reales, lo que permite medir distancias y velocidades en unidades físicas (mm, mm/s).

5.8 Detección de eventos

Sobre las trayectorias en milímetros aplicamos **reglas** inspiradas en los árbitros automáticos (*Auto-Refs*) de la liga RoboCup SSL. Detectamos 8 tipos de evento, cada uno con un umbral concreto:

Evento	Regla (simplificada)	Umbral
Kick (tiro/golpe)	el balón acelera de golpe	> 500 mm/s
Gol	el balón entra en el recuadro de una portería	dentro del bbox

Evento	Regla (simplificada)	Umbral
Pase	el balón cambia de poseedor del mismo equipo	> 300 mm recorridos
Intercepción	el balón cambia de poseedor del equipo rival	cambio de dueño
Retención	un robot retiene el balón demasiado tiempo	> 1.5 s a < 90 mm
Colisión	dos robots se tocan	< 50 mm entre centros
Sin progreso	el balón casi no se mueve por mucho tiempo	< 50 mm en 5 s
Robot dañado	un robot casi no se mueve por mucho tiempo	< 20 mm/s por 60 s

La detección de gol merece una nota: en lugar de inventar una “zona de gol” teórica, usamos el **recuadro real de la portería de color** (amarilla o azul) que detectamos en la calibración. Cuando el balón cae dentro de ese recuadro, contamos un gol (con un anti-rebote de 3 segundos para no contar el mismo gol varias veces).

5.9 Estadísticas en tiempo real

Mientras procesamos, una clase llamada `MatchStats` lleva la cuenta de:

- **Marcador** (goles del equipo A y B).
- **Posesión**: qué porcentaje del tiempo el balón estuvo más cerca de un robot de cada equipo.
- **Distancia recorrida y velocidad máxima** de cada robot y del balón.
- **Conteo de cada tipo de evento**.

5.10 Visualizaciones

Generamos cinco tipos de salida visual para “contar la historia” del partido:

1. **Video anotado** (`annotated.mp4`): el video original con los recuadros de los robots (coloreados por equipo), el balón, las porterías, un **banner superior** persistente con marcador, posesión y tiempo, y un banner inferior con los eventos recientes.
2. **Mapas de calor** (`heatmaps`): muestran las zonas donde más actividad hubo, uno global y uno por equipo.
3. **Trayectorias** (`trails`): las rutas completas de cada robot y del balón sobre una vista cenital de la cancha.
4. **Diagrama de Voronoi**: divide la cancha en regiones según qué robot está más cerca de cada zona, mostrando el “control del espacio”.
5. **Dashboard HTML interactivo**: una página web autocontenida (con la librería Plotly) que reúne el marcador, la línea de tiempo de eventos, la posesión y las trayectorias, navegable en cualquier navegador.

6. ¿Qué es fine-tuning y LoRA? (concepto base)

La categoría Profesional exige **innovar** sobre SAM 3. Una de las formas más potentes es el **fine-tuning** (ajuste fino).

6.1 Fine-tuning

SAM 3 ya sabe segmentar objetos en general, pero nunca vio específicamente nuestros robots de fútbol (que son pequeños, $\sim 30 \times 30$ píxeles, vistos de lado y con desenfoque). El **fine-tuning** consiste en seguir entrenando un poco el modelo con ejemplos de **nuestro** dominio para que mejore en esa tarea concreta.

El problema: SAM 3 tiene **cientos de millones de parámetros**. Reentrenarlos todos sería lentísimo y requeriría una tarjeta gráfica enorme.

6.2 LoRA: ajuste fino eficiente

LoRA (*Low-Rank Adaptation*, adaptación de bajo rango) es una técnica ingeniosa: en vez de modificar los millones de pesos originales (que se quedan **congelados**), le **agrega** unas pocas “matrices de ajuste” pequeñas en puntos estratégicos del modelo. Solo entrenamos esas matrices nuevas.

En nuestro caso, LoRA solo entrena **3.88 millones de parámetros**, que son el **0.46 %** del modelo total. Esto cabe sin problemas en nuestra tarjeta gráfica (RTX 5080, 16 GB) y entrena en minutos en vez de horas.

6.3 El problema de las etiquetas

Para hacer fine-tuning supervisado normalmente se necesitan **cientos de imágenes etiquetadas a mano** (marcar la silueta exacta de cada robot), lo que toma muchísimas horas humanas.

Nuestra solución fue la **pseudo- anotación**: usamos el propio SAM 3 base para generar las máscaras sobre 15 videos del torneo, nos quedamos solo con las de **alta confianza** ($\text{score} \geq 0.6$) que además pasaran los filtros geométricos, y usamos esas 524 máscaras curadas como “etiquetas” para entrenar el LoRA. Cero horas de etiquetado manual.

7. Las cuatro líneas de innovación

La convocatoria define cuatro líneas posibles de innovación sobre SAM 3 (§ 3.7.3). Cubrimos **las cuatro**:

1. **Prompts y contexto**: validamos empíricamente qué prompts funcionan mejor ("soccer robot" con 0.94 vs prompts elaborados con 0.34).
2. **Fine-tuning LoRA**: descrito arriba. Los **resultados cuantitativos** son contundentes (medidos con la métrica **IoU**, *Intersection over Union*, que mide qué tanto se solapan la máscara predicha y la correcta; va de 0 a 1, más alto es mejor):

Métrica	SAM 3 base	Con LoRA	Mejora
IoU global	0.046	0.912	+1882 %
IoU robots	0.049	0.956	+1839 %
IoU balón	0.036	0.780	+2059 %

El modelo afinado segmenta casi 20 veces mejor que el base en nuestro dominio, entrenando en **menos de 17 minutos** sobre RTX 5080.

3. **Integración con trackers**: combinamos SAM 3 con OC-SORT (robots) y un filtro de Kalman (balón), algo que el modelo base no hace por sí solo.
4. **Post-procesamiento geométrico**: la homografía, el cálculo de velocidades y posesión, y la detección de eventos por reglas estilo AutoRefs.

8. Resultados y validación

8.1 Calidad del software

- **83 pruebas automáticas** (*tests*) que verifican cada módulo; todas pasan.
- **Prueba de humo** (*smoke test*) que valida los 12 componentes del sistema en ~13 segundos.
- **Reproducibilidad**: fijamos las semillas aleatorias (*seeds*) y dejamos las versiones de todas las librerías ancladas, de modo que los resultados se pueden repetir.

8.2 Resultados sobre videos reales

Procesamos varios clips del torneo. Ejemplo del clip IMG_9821 (60 segundos): 186 eventos detectados, 4 goles, 30 robots seguidos, posesión 74 % / 26 %.

Una limitación honesta: en clips donde la cámara enfoca casi siempre a un solo equipo, la estimación de posesión se sesga (queda cerca de 100 % / 0 %), porque el sistema literalmente no ve al otro equipo el tiempo suficiente. Lo dejamos documentado como una limitación inherente a las grabaciones de espectador.

9. Cómo reproducir el proyecto

El repositorio público está en: <https://github.com/JAPerezC/futbotmx-ajolotesfc>

Pasos resumidos (detalle completo en el README.md):

1. Clonar el repositorio

```
git clone git@github.com:JAPerezC/futbotmx-ajolotesfc.git
cd futbotmx-ajolotesfc
```

2. Crear entorno de Python 3.12

```
py -3.12 -m venv .venv
.venv/Scripts/pip install -r requirements.txt
.venv/Scripts/pip install torch torchvision \
    --index-url https://download.pytorch.org/whl/cu128
```

3. Autenticarse en Hugging Face (SAM 3 es de acceso controlado)

```
.venv/Scripts/hf auth login --token <TU_TOKEN>
```

4. Verificar la instalación

```
.venv/Scripts/python -m pytest tests/ -q
.venv/Scripts/python scripts/smoke_test.py
```

5. Procesar un video

```
.venv/Scripts/python scripts/run_pipeline.py \
    --video data/raw/drive_samples/video-977.mov --stride 3
```

10. Glosario de términos

- **Píxel**: la unidad mínima de una imagen; un punto con un color.
- **RGB**: modelo de color por cantidades de Rojo, Verde y Azul.
- **HSV**: modelo de color por Matiz, Saturación y Brillo; mejor para identificar colores bajo iluminación variable.
- **Frame** (cuadro): cada imagen individual de un video.
- **fps**: cuadros por segundo (*frames per second*).
- **Stride** (paso): procesar 1 de cada N frames para ahorrar cómputo.
- **Segmentación**: asignar cada píxel a un objeto; produce una **máscara**.
- **Máscara**: imagen en blanco y negro que marca qué píxeles son del objeto.
- **Bounding box** (recuadro): rectángulo que encierra un objeto.
- **Tracking** (seguimiento): mantener la identidad de un objeto entre frames.
- **Track id**: número de identidad que se asigna a cada objeto seguido.
- **Homografía**: matriz 3×3 que corrige la perspectiva de la cámara y traduce de píxeles a coordenadas reales de la cancha.
- **Top-down** (cenital): vista desde arriba, como un plano.
- **SAM 3**: *Segment Anything Model 3*, el modelo de segmentación de Meta.
- **Prompt**: instrucción que se le da a SAM 3 (texto, caja o punto).
- **Vocabulario abierto**: capacidad de entender categorías descritas en lenguaje natural, no una lista fija.
- **Parámetros / pesos**: los números internos ajustables de un modelo.
- **Inferencia**: usar un modelo ya entrenado para predecir.
- **Fine-tuning**: seguir entrenando un modelo para una tarea específica.
- **LoRA**: técnica de fine-tuning que entrena solo unas matrices pequeñas añadidas, congelando el modelo original.
- **Pseudo-anotación**: generar etiquetas de entrenamiento automáticamente con el propio modelo en lugar de a mano.
- **IoU** (*Intersection over Union*): métrica de 0 a 1 que mide cuánto se solapan la máscara predicha y la correcta.
- **Kalman (filtro de)**: técnica que predice y corrige la posición de un objeto en movimiento, útil ante oclusiones.
- **OC-SORT**: algoritmo de seguimiento multi-objeto.
- **k-means**: algoritmo de agrupamiento que separa datos en k grupos; lo usamos con $k=2$ para separar los dos equipos por color.
- **Hough (transformada de)**: algoritmo para detectar líneas rectas en una imagen.
- **Envolverte convexa** (*convex hull*): el contorno más ajustado que rodea un conjunto de puntos sin concavidades.
- **Voronoi (diagrama de)**: partición del espacio en regiones según el punto más cercano; muestra control de territorio.
- **fp16 / fp32**: precisión numérica de 16 o 32 bits; fp16 es más rápida y ligera.
- **VRAM**: memoria de la tarjeta gráfica (GPU).
- **Pipeline**: cadena de etapas de procesamiento conectadas en serie.
- **AutoRef**: árbitro automático; sistema de reglas que detecta eventos del juego, inspirado en RoboCup SSL.